



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

#KubeCon #CloudNativeCon
#OpenInfraSummit #PyTorchCon

China 2026



Red Hat



Meta

vLLM-Helion

SOTA LLM Performance by Advanced Autotuning and Fine-Grained Dispatching

Yanan Cao, Meta

Xiaohong (Sean) Chen, Red Hat

Shangdi Yu, Meta



vLLM Kernel Optimization Landscape

- **torch.compile + TorchInductor**
 - Graph-level optimization and automatic compiler-generated fusion
- **Optimized kernel libraries**
 - GEMM, Attention, etc.
 - CUTLASS, DeepGEMM, FlashInfer
- **vLLM CustomOp**
 - CUDA / Triton on GPUs
- **Manual fusion**
 - Explicit operator fusion
 - Increasing preference for vendor-supported DSLs, e.g., NVIDIA CuTe DSL

What's Helion?

- **PyTorch-native kernel DSL**
 - Write concise, tile-based kernels in Python
- **Multiple compiler backends**
 - Triton, CuTe DSL, Pallas
 - One Helion kernel can generate many specialized kernels
- **Systematic autotuning**
 - Automatically explores implementation and config choices
 - Optimizes each kernel for its workload and hardware

```
def matmul(out, a, b):
    split_k = hl.register_tunable("split_k", ...)
    swap_ab = hl.register_tunable("swap_ab", ...)

    k_block_size = cdiv(K, split_k)

    for tile_m, tile_n, outer_k in hl.tile(
        [M, N, K],
        block_size=[None, None, k_block_size],
    ):
        acc = hl.zeros([tile_m, tile_n], torch.float32)
        acc_t = acc.t()

        for tile_k in hl.tile(outer_k):
            if swap_ab:
                acc_t = hl.dot(
                    b[tile_k, tile_n].T,
                    a[tile_m, tile_k].T,
                    acc=acc_t,
                )
            else:
                acc = hl.dot(
                    a[tile_m, tile_k],
                    b[tile_k, tile_n],
                    acc=acc,
                )

        out_blk = acc_t.T if swap_ab else acc

    if split_k == 1:
        out[tile_m, tile_n] = out_blk
    else:
        hl.atomic_add(out, [tile_m, tile_n], out_blk)
```

Simplified pseudocode: one Helion kernel for Standard, Split-K, and Swap-AB GEMM



KubeCon



CloudNativeCon



vLLM-Helion Value Proposition

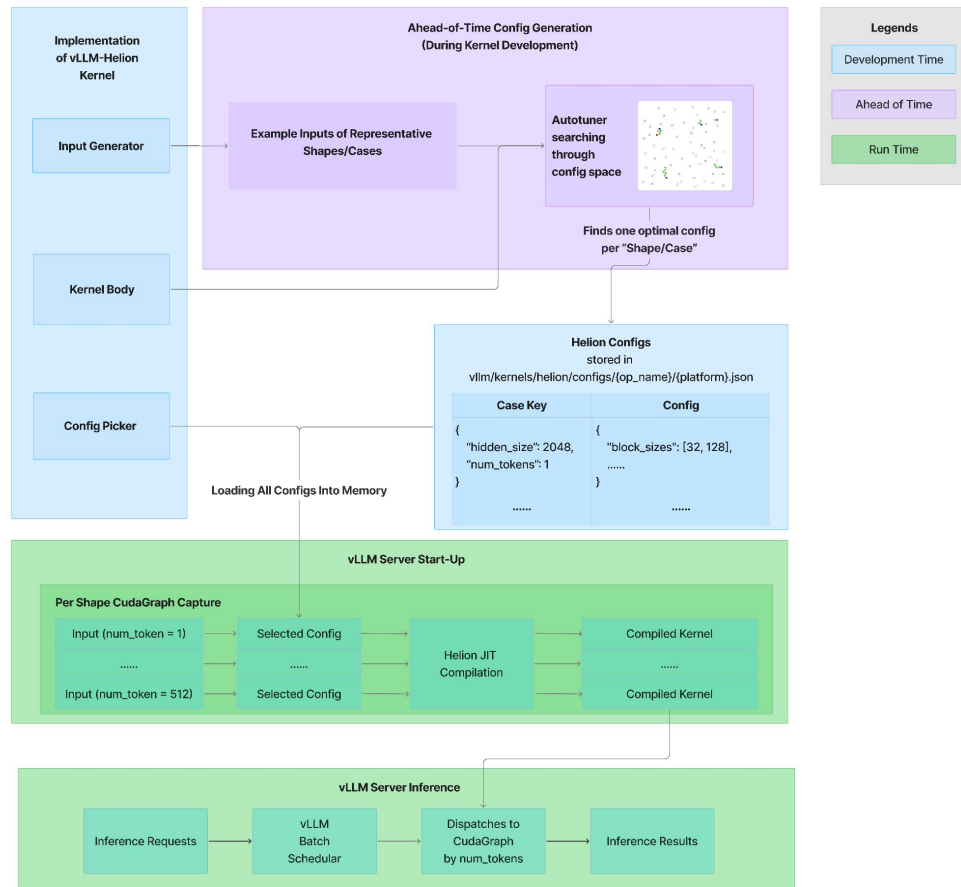
- **Performance**
 - SOTA performance potential across workloads and hardware
- **Systematic Tuning**
 - Systematic per-shape tuning across diverse input shapes
 - LLM-guided search combines LLM reasoning with systematic tuning
- **Abstraction & Portability**
 - One kernel implementation across workloads and hardware
 - Tunable control flow and algorithmic choices enable generic kernels
- **Client-Side Optimization**
 - Tune kernels for specific models, workloads, and deployments

vLLM-Helion Design Principles

- **Maximize Performance**
 - Systematic, workload-aware autotuning
- **Preserve Developer Experience**
 - One concise kernel implementation
 - Hide specialization and tuning complexity
- **Integrate Naturally with vLLM**
 - Follow vLLM CustomOp conventions
 - Work with existing benchmarking and runtime infrastructure

Architecture Overview

- **Kernel Implementation & Harness**
 - **Helion kernel**: algorithm implementation.
 - **Input generator**: define representative tuning/benchmark cases.
 - **Config picker**: map runtime inputs to tuned configs.
- **Autotuning Component**
 - Autotunes kernels to produce optimal configs for each shape
- **Config Management Component**
 - Handles config storage and retrieval
 - Tuned configs stored as JSON files in the vLLM codebase
- **Runtime Dispatch Component**
 - Low-overhead config selection and dispatch



Kernel Authoring

Helion Kernel Body

- Register the kernel with the `@register_kernel` decorator
- High-level, PyTorch-like kernel implementation
- Tile-based programming model
- High-level PyTorch ops supported
- Typically consists of a host function and device loop

```
@register_kernel(
    config_picker=pick_config,
    input_generator=generate_inputs,
)
def silu_mul_fp8(input: torch.Tensor, scale: torch.Tensor) -> torch.Tensor:
    assert scale.numel() == 1, "Scale must be a scalar Tensor"
    d = hl.specialize(input.shape[-1]) // 2
    inp_2d = input.view(-1, d * 2)

    out = torch.empty_like(inp_2d[:, :d], dtype=torch.float8_e4m3fn)
    inv_scale = 1.0 / hl.load(scale, [0])

    for tm, tn in hl.tile([inp_2d.shape[0], d]):
        a, b = inp_2d[tm, tn], inp_2d[tm, d + tn]
        res = torch.nn.functional.silu(a) * b
        out[tm, tn] = (res.float() * inv_scale).to(out.dtype)

    return out.view(input.shape[:-1] + (d,))
```

Kernel Authoring

Input Generator

- Defines representative inputs for each kernel
- Drives automated benchmarking and autotuning
- Controls which input dimensions to specialize and tune

```
def generate_inputs():  
    sizes = [2048, 2880, 4096, 8192, 11008, 14336]  
    tokens = [1, 2, 4] + list(range(8, 256, 8)) + list(range(256, 513, 16))  
    inputs = {}  
  
    for t in tokens:  
        for s in sizes:  
            x = torch.randn(t, 2 * s, device="cuda", dtype=torch.bfloat16)  
            scale = torch.tensor([1.0], device="cuda")  
            inputs[CaseKey({"intermediate": s, "numtokens": t})] = (x, scale)  
  
    return inputs
```


Kernel Authoring

Config Picker

- Maps runtime inputs to a representative tuning case
- Selects the corresponding tuned config
- Handles inputs without an exact match

```
def pick_config(args, keys):
    if not keys: return None

    x, _ = args
    isize = x.shape[-1] // 2
    ntokens = x.view(-1, x.shape[-1]).shape[0]

    cache_key = (ntokens, isize)
    if cache_key in _pick_cache:
        return _pick_cache[cache_key]

    by_isize = {}
    for k in keys:
        if not k.is_default():
            by_isize.setdefault(k["intermediate"], []).append(k["numtokens"])

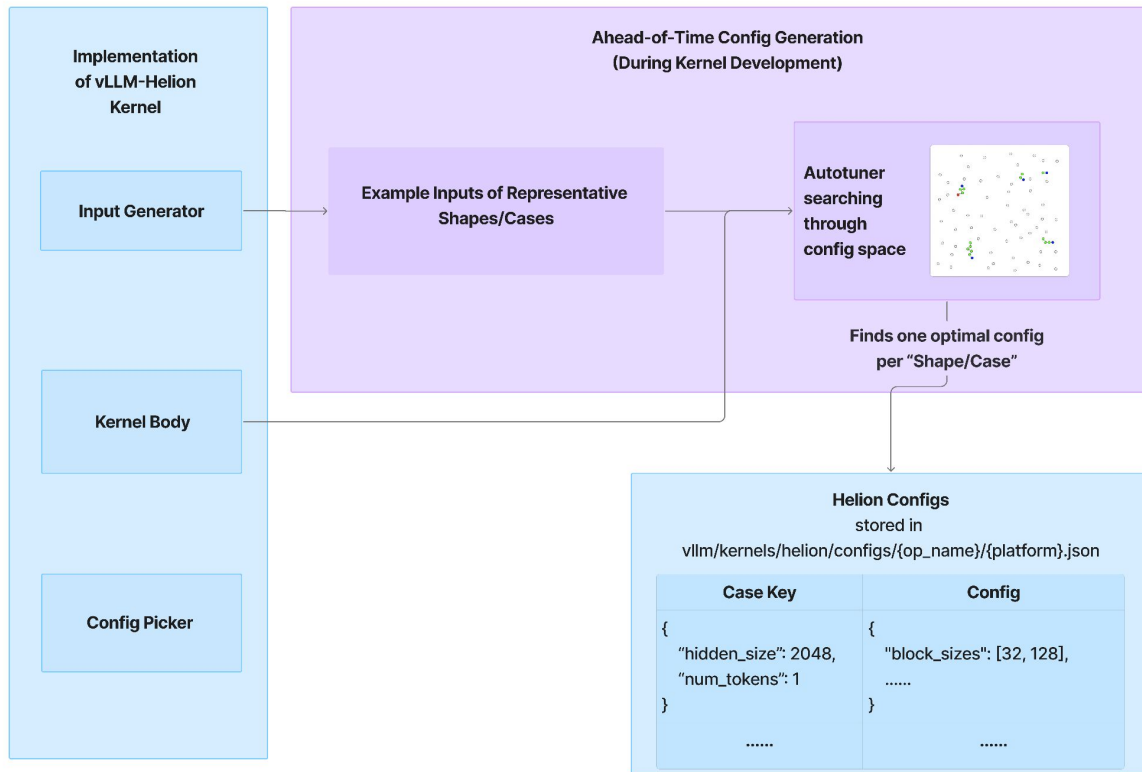
    if not by_isize: return None

    best_isize = min(by_isize, key=lambda s: abs(s - isize))
    avail = sorted(by_isize[best_isize])
    best_nt = next((n for n in avail if n >= ntokens), avail[-1])

    res = CaseKey({"intermediate": best_isize, "numtokens": best_nt})
    _pick_cache[cache_key] = res
    return res
```

Autotuning

- Iterate over input cases defined by the Input Generator
- Search low-level kernel configs and algorithmic choices
- Benchmark candidates and select the best-performing config
- Full tuning can take hours to days



Configs Management

- Each kernel may have dozens to hundreds of tuned configs
- Tuned configs are stored as JSON files in the vLLM codebase

Helion Configs

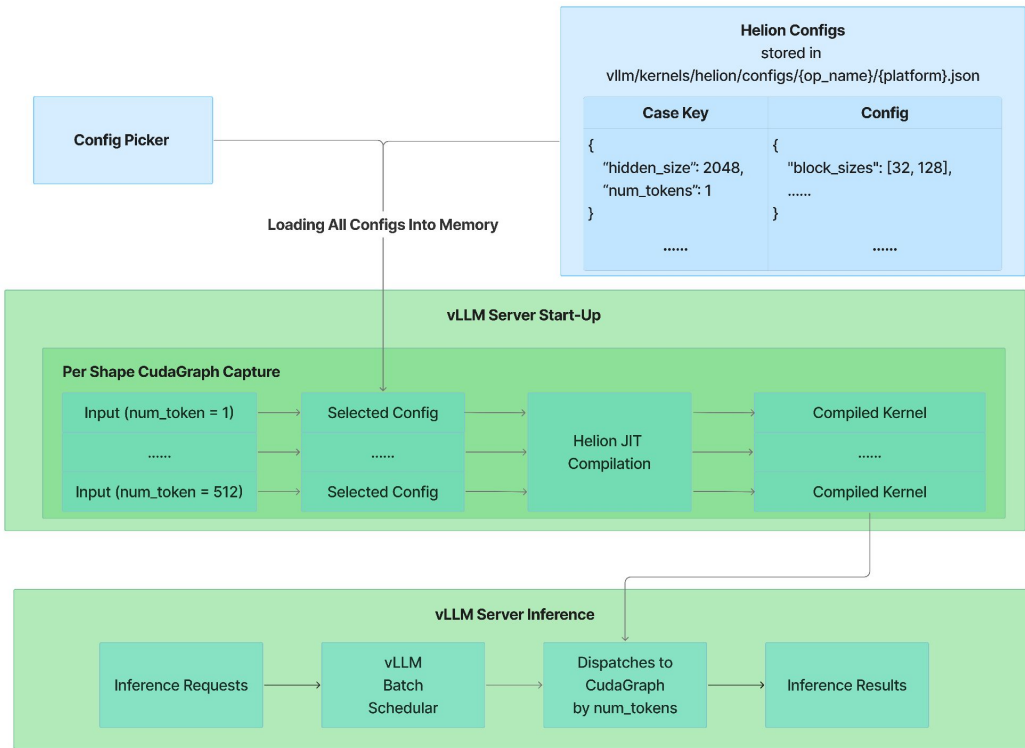
stored in

vllm/kernels/helion/configs/{op_name}/{platform}.json

Case Key	Config
{"hidden_size": 2048, "num_tokens": 1}	{ "block_sizes": [32, 128], "loop_orders": [[0, 1]], }
{"hidden_size": 4096, "num_tokens": 1}	{ "block_sizes": [1, 128], "loop_orders": [[0, 1]], }
{"hidden_size": 2048, "num_tokens": 2}	{ "block_sizes": [32, 128], "loop_orders": [[1, 1]], }
{"hidden_size": 4096, "num_tokens": 2}	{ "block_sizes": [1, 128], "loop_orders": [[1, 1]], }
.....	
{"hidden_size": 2048, "num_tokens": 512}	{ "block_sizes": [32, 128], "loop_orders": [[0, 1]], }
{"hidden_size": 4096, "num_tokens": 512}	{ "block_sizes": [128, 128], "loop_orders": [[0, 1]], }

Runtime Dispatch

- vLLM captures per-num-token CUDA Graphs
- Config Picker selects the tuned config for each input case
- Helion kernels are JIT-compiled during graph capture
- Inference executes through pre-captured CUDA Graphs



vLLM-Helion Challenges: Performance

- Performance
- Performance
- Performance

Going head-to-head with highly optimized kernel libraries is challenging, but achievable.



KubeCon



CloudNativeCon



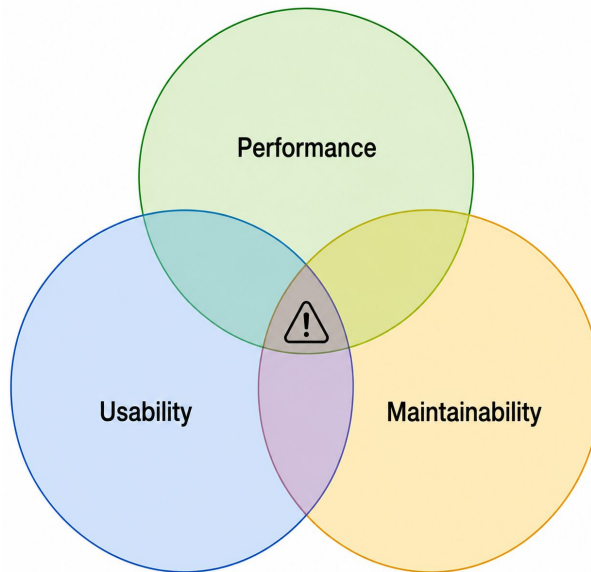
OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

vLLM-Helion Challenges: Overhead

- **AOT Tuning Overhead**
 - Hours of tuning as coverage grows
- **Startup Time Overhead**
 - JIT compilation during CUDA Graph capture increases cold-start time
- **Runtime Overhead**
 - CPU dispatch/launch overhead outside CUDA Graphs
- **Maintenance Overhead**
 - Pre-tuned configs add maintenance and validation burden



More fine-grained tuning improves performance, but increases either autotuning or upstream maintenance cost.

Adoption Strategy

Latency-critical kernels (e.g., Linear, Attention)

- **Existing Alternatives:**

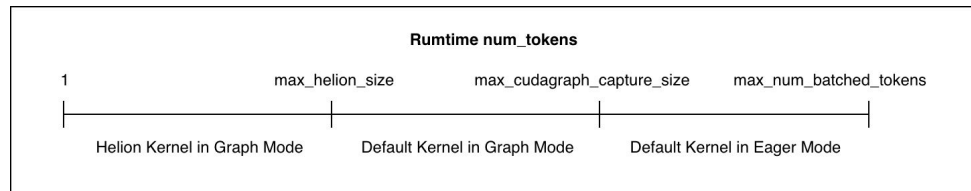
- Highly-optimized libraries: CUTLASS, DeepGEMM, and FlashInfer

- **Strategy:**

- Per-shape config tuning
- Use Helion for small decoding workloads
- Fall back to default kernels for larger workloads

- **Why:**

- Maximize performance
- Eliminate runtime overhead
- Reduce config tuning cost



Helion linear backend hybrid dispatch strategy

Adoption Strategy

Auxiliary kernels (e.g., quantization, activation)

- **Existing Alternatives:**
 - vLLM-maintained kernels
- **Strategy:**
 - Representative-shape config tuning
 - Use Helion within the CUDA Graph-captured range
 - Fall back to default kernels otherwise
- **Why**
 - [Initial study](#) shows that as few as 6 configs can deliver meaningful performance gains

Benchmark Results

- **Helion linear kernels**

- **Hopper:** Triton backend outperforms CUTLASS and DeepGEMM with per-shape tuned configs
- **Blackwell:** [Initial results](#) show on-par performance with the CuTe DSL backend

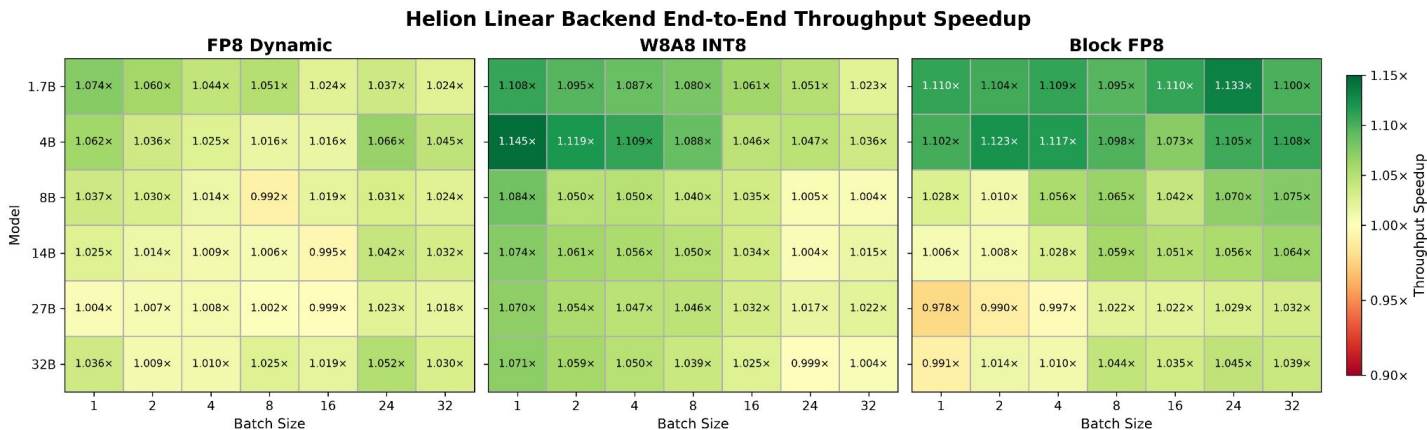
- **Helion auxiliary kernels**

- Triton backend outperforms torch.compile and vLLM CUDA kernels on both Hopper and Blackwell GPUs

Benchmark Results

Helion linear kernels

- GPU: H100 80GB HBM3
- Workload: ShareGPT
- Models: Qwen3 1.7B, 4B, 14B, 32B; Qwen3.8 2.7B
- Baseline: CUTLASS (W8A8_INT8, Dynamic_FP8) and FlashInfer + DeepGEMM (Block_FP8)
- **Takeaway:** Helion delivers consistent end-to-end performance gains, reaching over 10% for some workloads

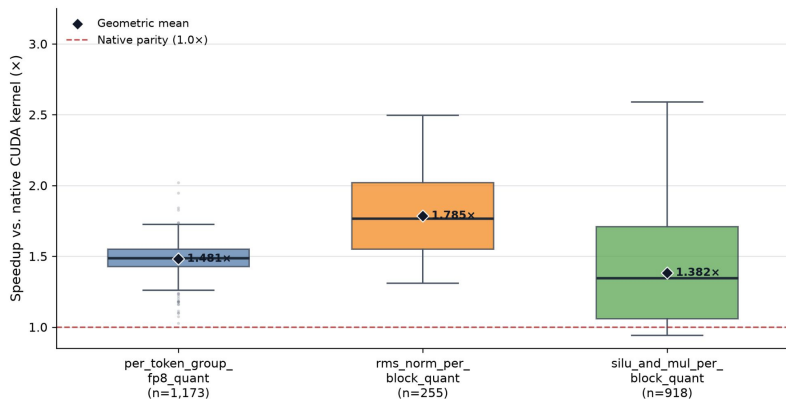


Benchmark Results

Helion auxiliary kernels

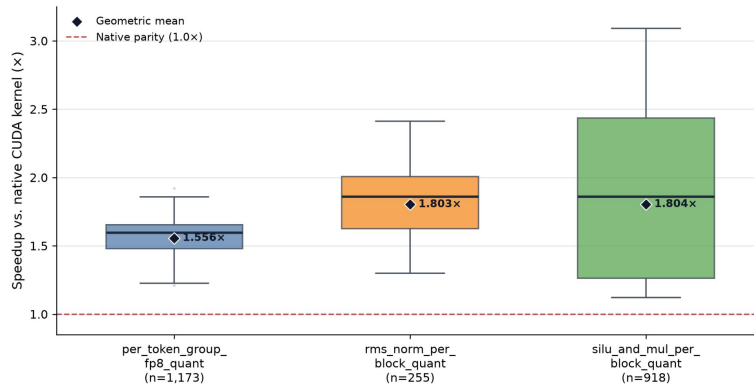
- Setup: a single config for each Helion kernel for H100, and up-to-six config for each Helion kernel for B200
- **Takeaway:** few configs can deliver meaningful performance gains

H100 Helion Kernel Speedup Distribution



Boxes show the interquartile range; center lines are medians; whiskers use 1.5x IQR.

B200 Helion Kernel Speedup Distribution

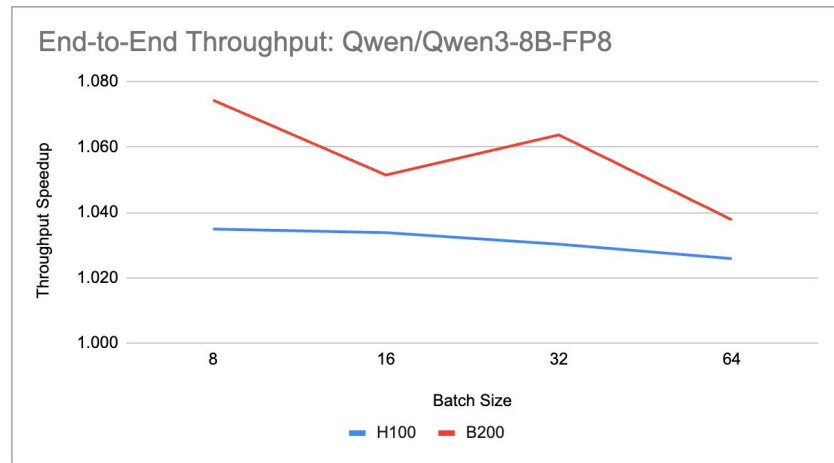


Boxes show the interquartile range; center lines are medians; whiskers use 1.5x IQR.

Benchmark Results

Helion auxiliary kernels

- GPUs: H100 and B200
- Helion kernels:
 - `per_token_group_fp8_quant`
 - `rms_norm_per_block_quant`
 - `rsilu_and_mul_per_block_quant`
- Configs:
 - One config for H100
 - Up-to-six configs for B200
- **Takeaway:** enabling a few small Helion auxiliary kernels can deliver meaningful end-to-end performance gains



Linear backend: DeepGEMM (FP32 ceiling E8M0 scales) on H100; CUTLASS on B200 to ensure all three Helion auxiliary kernels are exercised.

Toward a Practical Adoption Model

Current Status

- Helion integration framework upstreamed
- Autotuning & benchmarking tooling upstreamed
- Auxiliary kernels upstreamed
- Linear backend available in the [vLLM-Helion fork](#)

Two Adoption Paths

- Latency-Critical Kernels
 - Fine-grained tuning
 - Workload-specific autotuning by users
- Auxiliary Kernels
 - Representative-shape tuning
 - Ship kernels with as few as 6 pre-tuned configs

Try it. Tune it. Deploy it. Tell us what works and what doesn't.

→ *Feedback & contact on the last slide*

Recent & Ongoing Work

- **Better Performance**
 - Helion Core optimizations across backends
- **More Backends Support**
 - Cute DSL: Nvidia GPUs
 - [Pallas](#): TPU
- **Broader Kernel Coverage**
 - Auxiliary kernel integrations
 - Helion MoE backend
- **Faster autotuning**
 - [LLM-guided autotuning](#)
 - Better search heuristics
- **User engagement**

Resources

vLLM-Helion

- Kernel registration framework RFC: <https://github.com/vllm-project/vllm/issues/32219>
- Helion linear backend RFC: <https://github.com/vllm-project/vllm/issues/46526>
- Helion CustomOp RFC: <https://github.com/vllm-project/vllm/issues/53788>
- vLLM-Helion fork: <https://github.com/redhat-et/vllm-helion>

Learn More About Helion

- Blog: <https://pytorch.org/blog/project/helion/>
- Code: <https://github.com/pytorch/helion>
- Docs: https://helionlang.com/helion_tutorials.html
- Community: **Join us** on the [GPU MODE Discord](#) in the #helion channel



vLLM-Helion: SOTA LLM Performance by advanced autotuning and fine-grained dispatching



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE